

ResumeIQ Implementation Plan

Version 2.0 · 2026-09-29 · Revised from v1.0 (2026-09-17) · Target hardware: Intel i7 · NVIDIA RTX 4060 8 GB · 24 GB RAM

What Changed in v2.0

This revision keeps every feature, constraint, phase, metric and risk from v1.0 and adds the following. Items marked **[NEW]** are new phases or modules, **[EXTENDED]** marks v1.0 phases whose scope grew, and **[FUTURE SCOPE]** marks post-launch work that is fully planned here but not on the critical path.

Feature	Where it lives	Uses LLM	Uses RAG
Readability Reviewer (ATS 2.0) — a new Recruiter Readability Index (RRI), separate from the ATS score	Phase 10 [NEW]	Yes — skim-test simulation, bullet classification	Yes — role/seniority writing norms
General recommendations without a JD, including AI-likeness and lorem-ipsum/placeholder detection	Phase 9 [EXTENDED], Phase 11 [NEW]	Yes — synthesis & factual rewrites	Yes — inferred-role norms & guidance
JD-based feedback on how to improve the resume	Phase 12 [NEW]	Yes — section-level feedback	Yes — skill relationships, JD archetypes
AI-assigned job-match percentage	Phase 12 [NEW]	Yes — per-requirement verdicts (LLM-as-judge)	Yes — skill taxonomy/relations
Hardware-specific resource budget for the i7 / RTX 4060 8 GB / 24 GB RAM laptop	Section 3 [NEW], Phases 5 & 7	—	—
LLM & RAG placement map (which component is deterministic, embedding, RAG or LLM)	Section 5 [NEW]	—	—
Future: live matching JDs from trusted job sites	Phase F1 [FUTURE SCOPE]	Top-N re-scoring only	Local index of fetched JDs
Future: JD-based practice test built from a company's reported question history	Phase F2 [FUTURE SCOPE]	Yes — generation & short-answer grading	Yes — company question-history corpus

1. Executive Summary

ResumeIQ is a privacy-preserving, local-first web application that analyzes a candidate's resume, with or without a job description (JD). This plan turns the product specification into an ordered, buildable sequence of 16 core phases plus 2 future-scope phases, each with concrete tasks, technology choices, the role (if any) of the LLM and RAG layers, and exit criteria.

The system rests on one architectural principle every phase must protect: **separation of concerns between deterministic and probabilistic components**.

- **ATS compatibility** is calculated by explicit, weighted, testable rules — never by asking an LLM to guess a score.
- **Readability (ATS 2.0)** is a second, independent framework that measures how quickly and convincingly a *human* reviewer understands the resume. It combines deterministic text/layout

metrics, RAG-retrieved role norms, and constrained LLM judgments aggregated by a fixed, versioned formula.

- **Job-match quality** is reported two ways: a hybrid score (exact skill matching, taxonomy matching, sentence embeddings) and an **AI Match %** produced by the local LLM acting as an evidence-bound judge on each JD requirement. The hybrid score acts as a cross-check on the AI percentage.
- **The local LLM** is used for grounded explanation, AI-assigned requirement verdicts, readability judgments, recommendation synthesis, JD-specific rewriting, and AI-likeness explanations — always constrained to evidence present in the resume, the JD, or retrieved knowledge-base chunks, never inventing metrics, tools, or achievements.
- **RAG** grounds everything that needs outside knowledge: role/skill relationships, role-specific writing norms, project archetypes, and (future) company question history.

Everything the system tells the user must be traceable to a rule, a retrieved knowledge-base chunk, or a specific span of text in the resume or JD.

2. Product Goals & Non-Negotiable Constraints

2.1 Goals

Core analysis (from v1.0)

- Reproducible ATS compatibility score with an itemized, per-category breakdown.
- Detection of parsing/structural risk: columns, tables, text boxes, images, headers/footers, unusual headings, broken extraction, malformed links, inconsistent dates.
- Extraction of skills, technologies, roles, education, experience, projects, certifications and achievements into a canonical schema.
- Detection of unsupported/weak claims, timeline inconsistencies, duplicated content, vague bullets, missing evidence.
- "AI-likeness / generic writing" signals — never a certainty claim of AI authorship.
- JD-specific rewrite suggestions that preserve factual truth and never invent metrics, tools, or achievements.
- RAG-grounded role/skill/project/learning recommendations, prioritized by highest-value JD gaps.
- Full local execution on an i7 laptop with an RTX 4060 (8 GB VRAM) and 24 GB system RAM.
- A polished website with an interactive dashboard and an editable-suggestions workflow.

Added in v2.0

- **Readability Reviewer (ATS 2.0):** a Recruiter Readability Index that answers "will a human reviewer understand and be persuaded by this resume in the first skim?" — not just "can a parser read it?"
- **General recommendations (no JD):** a complete improvement report from the resume alone, including AI-likeness signals and detection of lorem ipsum, template placeholders and leftover boilerplate.
- **JD-based recommendations:** prioritized, section-by-section feedback on how to improve the resume for a specific JD.
- **AI Match %:** a job-matching percentage assigned by the local LLM, grounded in per-requirement evidence and cross-checked against the hybrid score.
- **Future scope:** live matching JDs from trusted job sources, and JD-based practice tests built from a company's reported question history.

2.2 Non-Goals / Safety Constraints (apply to every phase below)

- Never present the system as a hiring decision-maker or an objective measure of candidate worth.
- Never use protected/sensitive personal attributes for matching or scoring, and never infer demographic characteristics from names, photos, or other personal data.
- Never fabricate work experience, metrics, certifications, technologies, or achievements — every generated claim must trace to resume/JD evidence or an explicit user input.
- Never label text as definitively AI-generated; only "AI-likeness / generic writing signals" with confidence bands and reasons.
- Never claim the ATS score, the Readability Index, or the AI Match % is a universal industry standard — always disclose each as the application's own transparent, versioned rubric.
- Never let a JD, resume content, retrieved web text, fetched job postings, or question-bank content override system instructions — treat all uploaded and retrieved content as untrusted data, not instructions.
- **New:** Never present an AI Match % without its per-requirement evidence and confidence band, and never present it as a prediction of hiring outcome.
- **New (future scope):** Never send the resume or any personal data off-device for live job search — only derived, non-identifying search terms, and only after explicit opt-in.
- **New (future scope):** Never reproduce copyrighted or confidential interview questions verbatim; never present practice tests as the questions the candidate will actually receive.

These constraints should be encoded as automated checks (unit tests, prompt-injection tests, output validators) starting in Phase 0, not left as review-time judgment calls.

3. Target Hardware & Resource Budget

All models run on-device. The binding constraint is the 8 GB of VRAM on the laptop RTX 4060; 24 GB of system RAM is comfortable. The budget below keeps at least ~1 GB of VRAM free for the display/driver and KV-cache growth. Figures are planning estimates and are replaced by measured numbers in Phases 5 and 7.

Component	Recommended choice	Runs on	Approx. memory
Primary LLM	Qwen2.5-7B-Instruct or Llama 3.1 8B Instruct, 4-bit (Q4_K_M GGUF) via Ollama or llama.cpp; Qwen3 8B and Mistral 7B as benchmark alternatives	GPU	~4.5-5 GB weights + ~0.5-1 GB KV cache at 8k context
Fallback LLM	Qwen2.5-3B-Instruct or Llama 3.2 3B, 4-bit	GPU	~2-2.5 GB
Embedding model	bge-small-en-v1.5 (default) or bge-base-en-v1.5	GPU (fp16) or CPU	~0.2-0.5 GB
Reranker	bge-reranker-base (cross-encoder)	CPU by default, GPU if headroom	~0.5-1 GB
Vector DB	FAISS (flat/IVF) or Chroma, persisted to disk	CPU / RAM	<1 GB for the KB; ~1-2 GB with a live-job index (F1)
Backend + parsers + DB	FastAPI, PyMuPDF, python-docx, SQLite	CPU / RAM	~1.5-2.5 GB
Browser + frontend dev server	React + Vite	CPU / RAM	~2-4 GB

Component	Recommended choice	Runs on	Approx. memory
Total			VRAM ≈ 6-7 GB of 8 GB · RAM ≈ 8-12 GB of 24 GB

Operating rules that keep this budget safe

- Serialize LLM calls through a single-worker job queue (Ollama OLLAMA_NUM_PARALLEL=1, or one llama.cpp slot); batch requirement verdicts into as few calls as possible.
- Cap context at 8k tokens by passing structured JSON, selected evidence spans and top-k reranked chunks — never whole documents.
- Use constrained decoding (Ollama JSON-schema format or llama.cpp GBNF grammars) for every machine-read LLM output so parsing never depends on free text.
- Stream long generations to the UI (Server-Sent Events) so a 10-30 s analysis feels responsive.
- If VRAM pressure is detected, move the embedder/reranker to CPU first, then drop to the 3B fallback, then shorten context — in that order.

4. System Architecture

```

flowchart TD
    A[Browser] --> B[React + Tailwind frontend]
    B --> C[FastAPI backend + job queue]
    C --> D[File validation / sandbox]
    C --> E[PDF / DOCX / LaTeX ingestion]
    C --> F[Resume parser]
    C --> G[JD parser]
    C --> H[ATS rule engine]
    C --> R[Readability Reviewer - ATS 2.0]
    C --> Q[Quality & placeholder / lorem-ipsum checks]
    C --> I[Semantic matcher - hybrid score]
    C --> X[AI Match judge]
    C --> J[RAG retriever + reranker]
    C --> K[Local LLM service]
    C --> L[Rewrite / output validator]
    C --> M[Recommendation engine - general & JD-based]
    C -.future.-> LJ[Live JD connector]
    C -.future.-> AS[Assessment engine]
    H --> N[(SQLite / PostgreSQL)]
    J --> O[(Local vector DB)]
    K --> P[RTX 4060 local LLM]
    R --> J
    R --> K
    X --> K
    X --> I
    M --> J
    M --> K
    LJ -. -> O
    AS -. -> J
  
```

Backend stack: Python + FastAPI, modular by responsibility (ingestion, parsing, nlp, ats, readability, quality, matching, rag, llm, rewriting, recommendations, evaluation; plus jobs and assessments for future scope) — mirrored 1:1 in the repository layout.

Frontend stack: React + Tailwind, talking to the backend only through the versioned REST API in Section 8.

Storage: SQLite for local/dev, PostgreSQL-compatible for later deployment; a local vector database (FAISS or Chroma) for embeddings, kept separate from the relational store so it can be swapped independently.

LLM layer: runs entirely on-device (Ollama or llama.cpp serving a 4-bit quantized 7B–8B model) so no resume or JD content leaves the machine. The module boundary between the LLM service and everything else must stay strict enough that the model can be swapped without touching the deterministic ATS, readability-metric or extraction layers — this is a hard architectural requirement, not an aspiration. All prompts live in versioned template files; every LLM output records model name, quantization, prompt version and seed.

5. LLM & RAG Placement Map [NEW]

This map is the contract for where probabilistic components are allowed. A cell reading "—" means that layer must **not** be used for that feature. Wherever the LLM contributes to a number, the number is produced by a deterministic aggregation of structured LLM outputs, never by asking the model for a final score directly.

Feature	Deterministic	Embeddings	RAG	LLM
ATS score	All scoring rules	—	—	Plain-language explanation of rule results only
Readability (ATS 2.0)	Clarity, concision, hierarchy, timeline metrics; final aggregation	Bullet redundancy, section coherence	Role/seniority writing norms, guidance chunks	Skim-test simulation; bullet impact classification on an anchored rubric
Placeholder / lorem ipsum	Lexicon + regex + template fingerprints	Similarity to known template text	—	Confirms ambiguous semantic placeholders only
AI-likeness signals	Stylometric features	Template/cliché similarity	—	Explanation + factual rewrite
General recommendations (no JD)	Finding collection & priority ranking	Target-role inference	Role norms, skill relations, guidance	Synthesis of prioritized, cited recommendations
Hybrid match score	Exact/normalized skill match, weights	Semantic similarity	Skill taxonomy	—
AI Match %	Verdict aggregation, cross-check vs hybrid	Evidence retrieval per requirement	Skill relationships	Per-requirement verdicts (LLM-as-judge)
JD-based feedback	Gap classification, uplift ranking	Requirement-evidence alignment	Skill relations, role archetypes	Section-level feedback text
Rewriting	Hard fact validator	—	JD requirements, guidance	2–3 grounded candidates
Project recommendations	Gap clustering	—	Project archetypes	Tailored recommendation text
F1 Live JDs	Filters, dedupe, trust allowlist	Candidate retrieval	Local index of fetched JDs	AI Match % for top-N only
F2 Assessments	MCQ/code grading, timer	Question dedupe	Company question history	Question generation; short-answer grading

6. Repository Structure

```
resumeiq/
├── frontend/
│   ├── src/components/
│   ├── src/pages/
│   ├── src/api/
│   └── src/types/
├── backend/
│   ├── api/
│   ├── jobs/           # background job queue for LLM work (NEW)
│   ├── ingestion/
│   ├── parsing/
│   ├── nlp/
│   ├── ats/
│   ├── readability/   # ATS 2.0 / Recruiter Readability Index (NEW)
│   ├── quality/        # placeholder, lorem ipsum, weak-claim checks (NEW)
│   ├── matching/       # hybrid score + AI Match judge
│   ├── rag/
│   ├── llm/
│   │   └── prompts/    # versioned prompt templates (NEW)
│   ├── rewriting/
│   ├── recommendations/ # general/ and jd/ sub-modules
│   ├── evaluation/
│   ├── livejobs/       # future scope F1
│   └── assessments/    # future scope F2
├── knowledge_base/
│   ├── raw/
│   ├── processed/
│   ├── metadata/
│   ├── role_norms/     # readability & writing norms (NEW)
│   └── question_bank/  # future scope F2
├── models/
│   └── README.md
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── constraints/    # non-negotiable-constraint suite (NEW)
│   └── evaluation/
├── experiments/
├── docker/
├── docs/
└── README.md
```

Each backend/ subpackage owns exactly one pipeline stage and exposes a typed interface (Pydantic models) so later phases can be developed and tested against a stable contract even before the upstream stage is finished — this is what lets the phases below build incrementally instead of requiring a big-bang integration.

7. End-to-End User Workflow [EXTENDED]

- 1 User opens the site and selects Resume Analysis.
- 2 User uploads a PDF, DOCX, or .tex / .zip LaTeX project (or a compiled PDF plus optional source).
- 3 User optionally uploads or pastes a job description. **If none is given, the user may optionally pick a target role; otherwise the system infers the top-3 likely roles and asks the user to confirm one.**
- 4 System validates file type, size, and content (virus scan and sandboxed parsing in production).
- 5 Document ingestion extracts text, sections, layout metadata, links, and source information.
- 6 Resume parser builds a normalized structured profile.
- 7 JD parser builds a normalized job-requirement profile (JD path only).
- 8 Deterministic ATS engine calculates ATS compatibility.

- 9 **Quality checks run: placeholder/lorem-ipsum detection, weak/vague bullets, unsupported claims, timeline issues, AI-likeness signals.**
- 10 **Readability Reviewer computes the Recruiter Readability Index using role norms retrieved via RAG.**
- 11 Semantic engine compares resume evidence against JD requirements; **the AI Match judge produces the AI Match % with per-requirement verdicts** (JD path only).
- 12 RAG retrieves relevant role/skill/project knowledge.
- 13 Local LLM produces grounded feedback, rewrites, and explanations.
- 14 Recommendation engine produces **general recommendations (no JD) or JD-based feedback**, plus skill gaps and project suggestions.
- 15 Website displays scores (ATS, Readability, Hybrid Match, AI Match %), evidence, warnings, suggestions, and an editable resume-improvement view.
- 16 User accepts/rejects individual rewrite suggestions and exports a report.
- 17 *Future scope*: user opts in to see live matching JDs, and/or takes a practice test for the chosen JD.

This sequence is the backbone the phases build toward: each phase makes one or more of these steps real, in order, so there is always an end-to-end (even if partial) path from step 1 to step 16.

Implementation Phases

Phases are listed in build order. Phases 1–8 and 13–16 carry over every task from v1.0 (renumbered where needed); Phases 10–12 are new; Phase 9 is extended. A milestone view of the same sequence appears in Section 12.

Phase 0 — Foundations & Constraint Test Suite **[NEW]**

Tasks

- 1 Scaffold the repository per Section 6 and set up CI (lint, type-check, unit tests, and a smoke test hitting `/api/resume/upload`).
- 2 Collect/anonymize a small fixture set of resumes (clean, column-heavy, table-heavy, image-heavy, corrupt, **template-with-placeholders, lorem-ipsum-filled, heavily generic**) and JDs (terse, verbose, nested).
- 3 Stand up the non-negotiable-constraints suite under `tests/constraints/`: fabrication checks, prompt-injection checks, AI-likeness confidence-language checks, score-disclosure checks, protected-attribute checks. Tests that need later modules start as expected-failures and are switched on as phases land.
- 4 Confirm target-hardware access (i7 + RTX 4060 8 GB + 24 GB RAM); install NVIDIA driver/CUDA, Ollama or llama.cpp, and record a baseline of free VRAM with the desktop running.
- 5 Add the background job queue (single worker, SQLite-backed job table) and SSE progress endpoint that every LLM-using phase will reuse.

Exit criteria: CI green on an empty skeleton; fixture sets checked in with anonymization notes; constraint suite runs in CI; hardware baseline documented.

Phase 1 — PDF/DOCX Ingestion & Structured Extraction

Tasks

- 1 Stand up FastAPI skeleton, `POST /api/resume/upload`, request validation (file type/size/MIME sniffing).

- 2 Implement PDF extraction with PyMuPDF: text, page/section coordinates, column/table/image detection. **Also retain font sizes, bold/weight, bounding boxes and reading order — the Readability Reviewer (Phase 10) needs this layout metadata.**
- 3 Implement DOCX extraction with python-docx: paragraphs, headings, tables, hyperlinks, styles, core metadata.
- 4 Normalize both extraction paths into the canonical resume JSON schema (contact / summary / education / experience / projects / skills / certifications / achievements / sections / document_metadata / **layout_metadata**).
- 5 Persist source_spans (page/paragraph offsets) for every extracted field so later modules can cite evidence back to the original document.
- 6 Add a sandboxed parsing path (subprocess/isolated worker) as the production posture, even if local dev runs in-process.

Tech: PyMuPDF, python-docx, FastAPI, Pydantic schema validation.

LLM / RAG role: None. Extraction is fully deterministic.

Exit criteria: a standard-section PDF and DOCX both parse into the canonical schema with no critical extraction errors; every field in the output carries a source_span; layout metadata is present for every text block; malformed/corrupt files fail gracefully with a structured error, not a crash.

Phase 2 — Deterministic ATS Rule Engine & Dashboard

Tasks

- 1 Implement each ATS dimension as an independent, testable rule returning {score, max_score, severity, evidence, remediation} (table below).
- 2 Keep weights in configuration (not hardcoded) so they can be tuned after evaluation without a code change.
- 3 Build POST /api/analysis/run and GET /api/analysis/{id}/ats to expose overall score, category scores + evidence, severity-tiered warnings (critical/high/medium/low), a parser preview (what was actually extracted), formatting-risk explanation, and remediation steps.
- 4 Build the dashboard UI: overall score, category breakdown, warnings list, parser preview panel. **Reserve dashboard slots for the Readability Index, Hybrid Match and AI Match % cards.**
- 5 Unit-test every rule against hand-crafted resume fixtures (clean, column-heavy, table-heavy, image-heavy, corrupt).

Dimension	Weight	Checks
Text extractability	20	Extractable text, missing/garbled spans, page ordering, character corruption
Section structure	15	Standard headings, section boundaries, logical ordering, duplicate sections
Formatting safety	15	Columns, tables, text boxes, images, unusual layout, decorative excess
Contact information	10	Email, phone, links, name, basic validity
Standard headings	10	Recognizable Education, Experience, Projects, Skills, etc.
Date consistency	10	Chronology, overlapping periods, malformed/future dates
Skill visibility	10	Skills in appropriate sections, not buried in graphics
Links/contact validity	5	URL/email syntax and extraction
Parsing risk	5	Overall parser confidence, unresolved content

LLM / RAG role: None for scoring. The LLM may later (Phase 8) phrase explanations of rule results, but never alters a score.

Exit criteria: ATS score is reproducible for identical input; component scores sum exactly to the total; every warning links to concrete evidence and a remediation step; the UI clearly labels the score as the application's own rubric, not an industry standard.

Phase 3 — JD Parsing & Skill Normalization

Tasks

- 1 Build POST /api/jd/analyze: accepts PDF/DOCX/text/paste, extracts raw text, normalizes into the JD schema (title / company / location / seniority / required_skills / preferred_skills / responsibilities / education_requirements / certifications / domain_terms).
- 2 Build a skill-normalization layer: canonical skill names + aliases (e.g. "JS" → "JavaScript"), backed by the Skill entity (canonical_name, aliases, category).
- 3 Classify required vs. preferred requirements, extract responsibilities and domain terms as distinct fields (not one bag of keywords).
- 4 Preserve source_spans for every extracted JD field, same as the resume pipeline.
- 5 Unit-test against a diverse JD fixture set (terse JDs, verbose JDs, JDs with nested requirement lists).
- 6 **Add an optional LLM-assisted fallback for requirement classification on JDs where the rule-based classifier's confidence is low; its output is schema-constrained and every extracted requirement must map back to a JD span.**

LLM / RAG role: Optional, low-confidence fallback only (task 6). Rules remain primary so JD parsing stays fast and testable.

Exit criteria: JD requirement classification (required/preferred/responsibility/education/certification) reaches an acceptable precision/recall on the fixture set (baseline measured, not assumed); skill aliasing correctly merges obvious synonyms in a manual spot-check; LLM fallback never emits a requirement without a source span.

Phase 4 — Hybrid Semantic Matching & Evidence Tracing

Tasks

- 1 Implement the job-match scoring dimensions (table below). This produces the **Hybrid Match score**, which becomes the deterministic cross-check for the AI Match % in Phase 12.
- 2 Implement skill-match classification per requirement: **Direct** match (same normalized skill), **Related** match (semantically/taxonomically related, not equivalent), **Partial** match (evidence exists but doesn't fully satisfy), **Missing** (no evidence), **Unsupported claim** (appears in Skills section but nowhere else in the resume).
- 3 Combine exact/normalized skill matching + taxonomy/related-skill matching + sentence-embedding similarity (placeholder embeddings until Phase 5 delivers the real model) + rule-based experience/project relevance scoring.
- 4 Persist matched evidence spans (Evidence entity: analysis_id, source_type, span, claim, confidence) so every conclusion is traceable to resume or JD text.
- 5 Expose GET /api/analysis/{id}/match.

Dimension	Weight
Required skills	30
Preferred skills	10
Relevant experience	20

Dimension	Weight
Relevant projects	15
Responsibilities alignment	15
Education alignment	5
Semantic/domain relevance	5

LLM / RAG role: Embeddings only; no LLM. The hybrid score must stay reproducible.

Exit criteria: every major job-match conclusion in a test run has at least one linked evidence span; missing skills are visibly distinguished from related/partial matches in the API response and UI.

Phase 5 — Local Embedding Model & Vector Database

Tasks

- 1 Benchmark small local sentence-transformer/BGE-family embedding models on the target laptop (RTX 4060 8 GB + i7 + 24 GB RAM) for throughput and memory footprint, **both on GPU (fp16) and CPU, measured while the LLM is resident in VRAM.**
- 2 Stand up FAISS or Chroma as the local vector database; wrap it behind a stable internal interface so the choice can change later without touching callers.
- 3 Replace Phase 4's placeholder embeddings with the real model; re-run the semantic-relevance scoring.
- 4 Add embedding versioning/metadata so re-embedding after a model swap is a traceable operation, not a silent change.
- 5 Load-test with realistic resume/JD volumes to confirm acceptable latency on-device.

Tech: sentence-transformers, bge-small-en-v1.5 / bge-base-en-v1.5, FAISS or Chroma.

Exit criteria: chosen embedding model + vector DB combination meets a defined latency budget on the target hardware **with the LLM loaded concurrently**; embedding version is recorded alongside every stored vector.

Phase 6 — RAG Knowledge Base & Retrieval Evaluation [\[EXTENDED\]](#)

Tasks

- 1 Curate the corpus from authoritative sources: ESCO and O*NET for occupations/skills, selected technology documentation, and a hand-built project-archetype library. Cover occupation descriptions, skill definitions/relationships, alternative names/synonyms, technology categories, role-to-skill relationships, project archetypes, learning/resource metadata, and the application's own resume-writing guidance.
- 2 **Add a role-norms collection (knowledge_base/role_norms/) for the Readability Reviewer and general recommendations: per role family and seniority — expected sections and ordering, what evidence reviewers look for (e.g. publications for research roles, projects above experience for new graduates), typical tools, typical outcome types, and anti-patterns. Written in-house and cross-referenced to O*NET tasks / ESCO skills; no scraped resumes.**
- 3 Build the ingestion pipeline: load source documents → normalize/clean → chunk on semantic/document boundaries → attach metadata (source, role, seniority, skill, section, collection, date/version) → generate embeddings → store in the vector DB.
- 4 Implement retrieval: hybrid semantic + lexical (BM25) top-k, with a reranking step (bge-reranker-base) before anything reaches the LLM; support metadata filters (collection, role, seniority).

- 5 Ensure only relevant retrieved evidence (not whole documents) is passed downstream, and that every recommendation stores its source chunk IDs.
- 6 Version the corpus so retrieval behavior is reproducible and auditable.
- 7 Evaluate retrieval quality (Recall@K, groundedness) against a small hand-labeled query set before wiring RAG into the LLM phase — **including role-norm queries**.
- 8 Expose GET /api/knowledge/search for debugging/inspecting retrieval.

Exit criteria: retrieval returns relevant, source-attributed chunks for a representative query set; every knowledge chunk carries traceable source/version metadata; role-norm retrieval is evaluated separately; RAG is confirmed to ground recommendations, not merely answer free-text questions.

Phase 7 — Local LLM Integration [EXTENDED]

Tasks

- 1 Stand up the LLM runtime — Ollama or llama.cpp — as a service the backend calls internally, never exposed directly to the browser.
- 2 Benchmark candidate 4-bit-quantized instruct models on the target hardware: Qwen2.5 7B, Qwen3 8B, Llama 3.1 8B, Mistral 7B, **plus a 3B fallback**. Select based on measured latency (time-to-first-token and tokens/s), VRAM headroom at 8k context, **JSON-schema adherence rate**, and output quality on resume-feedback-style prompts — not on general benchmarks alone.
- 3 Design the context-construction strategy: pass structured resume/JD JSON, selected evidence spans, and retrieved RAG chunks — never entire raw documents — to keep context short and grounded.
- 4 **Build the shared LLM client: versioned prompt templates, schema-constrained outputs (JSON schema / GBNF), temperature 0 and fixed seeds for any output that feeds a score, retry-on-invalid-schema, and logging of model/quant/prompt-version per call.**
- 5 Add a "Settings / model / runtime" backend config and matching frontend page so the model/runtime is swappable without a redeploy.
- 6 Add basic prompt-injection resistance tests: confirm JD/resume/RAG content cannot override system instructions (untrusted content always wrapped in delimited data blocks).

Exit criteria: the selected model completes representative feedback-generation requests within an acceptable latency on the RTX 4060 / i7 / 24 GB target; peak VRAM stays below 7 GB; schema-valid output rate $\geq 98\%$ after retries; swapping models requires no change to upstream deterministic modules; injected instructions inside a resume or JD are not followed.

Phase 8 — Grounded Feedback & JD-Specific Rewriting

Tasks

- 1 Implement the rewrite workflow: identify weak bullet → retrieve relevant JD requirement(s) → identify factual evidence already in the resume → generate 2–3 rewrite candidates → validate every technology/metric/entity in each candidate against source evidence → highlight newly-introduced JD-aligned concepts → surface for accept/reject/edit.
- 2 Build the rewrite validator as a hard gate: any candidate introducing an unsupported fact (a metric, tool, or claim absent from the resume and not explicitly supplied by the user) is rejected before it reaches the user, never just flagged after the fact.
- 3 Build POST /api/rewrite returning: original bullet, why it's weak, relevant JD requirement, suggested rewrite, evidence used, new keywords/skills introduced, unsupported-claims check result.
- 4 Build the rewrite editor UI: side-by-side original/suggested comparison with accept/reject/edit controls. **When a bullet lacks a metric, prompt the user to supply one ("What was the result? e.g. time saved, users served") rather than letting the model guess.**

- 5 Wire grounded explanations (why a score or warning is what it is) through the same evidence-validation path, not a separate looser one.
- 6 **Support rewrites without a JD (general mode): target the inferred/confirmed role's norms from RAG instead of JD requirements.**

Exit criteria: no rewrite reaching the UI in testing contains a fact absent from resume evidence or explicit user input; every rewrite response includes its evidence and its unsupported-claims check result; general-mode rewrites pass the same validator.

Phase 9 — AI-Likeness, Generic-Writing & Placeholder / Lorem Ipsum Detection [EXTENDED]

9A. AI-likeness / generic-writing signals (from v1.0)

Tasks

- 1 Implement detection for each signal type — generic phrasing, low specificity, buzzword density, repeated sentence templates, low information density, and stylistic mismatch across sections — as a probabilistic signal, never a binary AI/human classifier.
- 2 For each flagged sentence, return: original text, signal type, confidence band, plain-language explanation, and a factual rewrite suggestion drawing only on resume/user-supplied facts.
- 3 Build a held-out evaluation set of human-written and AI-assisted resume bullets; report precision/recall or calibration — do not ship without this measurement.
- 4 Ensure UI copy never states or implies certainty about authorship; confidence bands and "signals, not verdicts" language are mandatory in every surface that shows this output.

9B. Placeholder & lorem ipsum detection (new)

Leftover template text is one of the most damaging and easiest-to-fix resume errors, so this check is deterministic-first and runs on every upload, with or without a JD.

Tasks

- 1 **Lorem ipsum detector:** a lexicon of standard lorem-ipsum pseudo-Latin tokens; flag any span where the ratio of lexicon tokens exceeds a threshold, and any known opening phrase anywhere in the document.
- 2 **Template placeholder detector:** regex + fingerprint rules for bracketed/brace tokens ([Company Name], {Role}, <Insert>), stock fillers ("Your Name", "City, State", "Company Name", "Job Title"), masked values (XX%, 000-000-0000, MM/YYYY), example contacts (@example.com, john.doe), and TODO/TBD/"insert here" markers.
- 3 **Template-residue detector:** identical bullets repeated across roles, default template section names/instructions ("Describe your responsibilities here"), and embedding similarity to a small library of known template sample text.
- 4 **LLM confirmation (ambiguous cases only):** for spans with medium detector confidence, ask the LLM a constrained yes/no-with-reason question ("Is this span instructional/placeholder text rather than a real claim?"). Deterministic high-confidence hits never need the LLM.
- 5 Every finding returns span, type, severity (lorem ipsum and unfilled contact fields are **critical**), and a fix instruction. Findings also feed the ATS "Parsing risk" and Readability "Clarity" dimensions.

Exit criteria: module output is measured against the held-out set with reported precision/recall or calibration; no UI or API text claims authorship certainty; placeholder/lorem-ipsum detection reaches ≥ 0.95 recall on the placeholder fixture set with a measured false-positive rate on clean resumes.

Phase 10 — Readability Reviewer (ATS 2.0): Recruiter Readability Index [NEW]

Concept

A conventional ATS score answers one question: *can a machine parse this document and find the keywords?* A resume can pass that test and still fail with the person who reads it next. The Readability Reviewer is a separate framework, the **Recruiter Readability Index (RRI)**, that answers a different question: *in the first skim, does a human reviewer correctly understand who this candidate is, what they have done, and why it matters for the role?*

It differs from ATS in three ways. First, it scores **meaning, not matching**: bullets are judged on whether they show an action, a concrete object, and an outcome, not on keyword presence. Second, it is **role-aware through RAG**: what "readable and convincing" means for a new-grad software engineer differs from a senior data scientist or a nurse, so role/seniority norms are retrieved from the knowledge base and every piece of feedback cites them. Third, it includes a **skim-test simulation**: the LLM is shown only what a reviewer realistically takes in during a first pass (the top of page one, headings, job titles, the first words of each bullet) and asked to state what it concludes. Comparing that snapshot with the full-resume profile exposes what the candidate's strongest evidence fails to communicate quickly.

Headline outputs: RRI score (0–100, versioned rubric) · "6-second snapshot" (what a skimmer concludes vs. what the full resume supports) · attention map overlay on the resume preview · per-bullet grade (A–D) with reasons · top-5 highest-impact fixes, each citing a resume span and a role-norm chunk. Works with or without a JD; with a JD, the snapshot is also compared against the JD's top requirements.

RRI dimensions

Dimension	Weight	How it is measured	Method
First-impression clarity (skim test)	20	Skim-view extract → LLM states inferred target role, seniority, top 3 strengths → compared with the full-profile answers and the target role/JD. Score = agreement on role, seniority and strengths.	Deterministic extract + LLM (structured) + deterministic comparison
Impact & evidence density	20	Share of bullets that contain action + concrete object + outcome/result; share with verifiable specifics. LLM classifies each bullet against an anchored rubric (A–D with written anchors and examples); deterministic cues (verbs, numbers, named entities) corroborate.	LLM rubric classification + deterministic cues
Specificity & concreteness	15	Named tools/systems/domains per bullet, concrete nouns vs. abstractions, inverse of Phase 9 generic-writing signals.	Deterministic (NER, lexicons) + Phase 9 signals
Clarity & concision	15	Bullet length distribution, sentence length, passive voice, filler words, tense consistency, undefined acronyms, placeholder findings from Phase 9B.	Deterministic
Visual hierarchy & scan flow	10	Heading consistency, font-size hierarchy, whitespace density, bullets per role, page count vs. seniority, most relevant section position. Uses Phase 1 layout metadata.	Deterministic (layout)
Narrative coherence	10	Career progression is legible (titles/scope grow or pivots are explained), skills listed are demonstrated somewhere, no contradictory dates, summary agrees with body.	Deterministic + embedding similarity; LLM only to phrase findings
Role-norm alignment	10	Retrieved role/seniority norms (expected sections, evidence types, ordering) are checked item by item against the resume.	RAG + deterministic checklist; LLM only for ambiguous items

Scoring rules that keep it trustworthy

- The LLM never outputs the RRI or a dimension score. It outputs structured labels (bullet grade, inferred role, strengths list) under a fixed rubric; the score is a deterministic function of those labels plus the deterministic metrics.
- LLM judgments run at temperature 0 with a fixed seed; the skim test is run 3 times with different fixed seeds and majority-voted, and disagreement lowers the reported confidence band.
- Weights, rubric anchors and prompt versions live in configuration, and the RRI version is stored with every result.
- The RRI is disclosed in the UI as the application's own rubric for human readability, validated against a small recruiter panel (task 7) — not an industry standard.

Tasks

- 1 Implement the skim-view extractor: top third of page one, all headings, job titles/companies/dates, and the first ~8 words of each bullet, using layout metadata and reading order.
- 2 Write the anchored bullet-grading rubric (A-D with definitions and 3-5 in-house examples per grade and role family) and store it in the role-norms collection.
- 3 Implement each deterministic metric as an independent, unit-tested function returning {score, max_score, evidence, remediation}, mirroring the ATS rule interface.
- 4 Implement the LLM tasks (skim-test inference, bullet classification) as schema-constrained calls through the Phase 7 client; batch all bullets into one or two calls to stay within the VRAM/latency budget.
- 5 Implement RAG retrieval of role norms filtered by role family + seniority (from JD, user choice, or inferred role) and the item-by-item role-norm checklist.
- 6 Build POST /api/readability/run and GET /api/analysis/{id}/readability; build the UI: RRI card, dimension breakdown, 6-second-snapshot comparison, attention-map overlay, per-bullet grades that link into the rewrite editor.
- 7 Validation study: 3+ recruiters/hiring managers independently rate 50-100 anonymized resumes for skim clarity and overall readability; report inter-rater agreement and Spearman correlation between RRI and mean human rating; tune weights on a split, report on held-out resumes.

Exit criteria: RRI is reproducible for identical input and configuration (same labels → same score); every RRI finding cites a resume span and, where role-specific, a role-norm chunk ID; RRI correlation with recruiter ratings on held-out resumes is measured and reported (target set after the baseline, e.g. $p \geq 0.5$), and exceeds the correlation of the ATS score with the same ratings.

Phase 11 — General Recommendation Engine (No JD Provided) [NEW]

When the user uploads only a resume, ResumeIQ still produces a complete, prioritized improvement report. It aggregates every JD-independent finding, infers what role the resume is aiming at, and uses RAG to ground advice in that role's norms.

Tasks

- 1 **Collect findings** from ATS (Phase 2), placeholder/lorem ipsum and AI-likeness (Phase 9), weak/vague bullets, unsupported claims, duplicated content, timeline inconsistencies, and the Readability Reviewer (Phase 10) into one normalized Finding list with span, category, severity and source module.
- 2 **Infer target role:** embed the resume summary/titles/skills and retrieve the nearest O*NET/ESCO occupations; show the top 3 with reasons and let the user confirm or choose another. The confirmed role drives all RAG retrieval below.

- 3 **Retrieve role norms and guidance** for the confirmed role/seniority (Phase 6 role-norms collection + writing guidance) and derive role-level gaps (e.g. "no evidence of testing practice, which is typical for backend roles").
- 4 **Prioritize deterministically:** $\text{priority} = \text{severity weight} \times \text{estimated impact} \div \text{effort}$, with fixed tables per finding type; critical items (lorem ipsum, unfilled placeholders, unparseable sections, missing contact info) always rank first.
- 5 **LLM synthesis:** for the top N findings, the LLM writes a short recommendation (what, why, how) using only the finding's evidence and retrieved chunks, and — where relevant — produces a Phase 8-validated rewrite. Output schema: {title, category, why, how, evidence_ids, source_ids, effort, impact, rewrite?}.
- 6 Build POST /api/recommendations/general and the UI "Improve my resume" view: grouped by Critical fixes / Readability / Content strength / Role alignment / AI-likeness signals, each item expandable to show evidence and sources.

Exit criteria: a resume with no JD yields a prioritized report in which every item links to a resume span and, if role-based, a RAG source ID; all placeholder/lorem-ipsum findings in the fixture set appear as critical items; no recommendation text contains an unsupported fact (validator + constraint suite).

Phase 12 — JD-Based Feedback & AI Match Percentage [NEW]

12A. AI Match % (percentage assigned by the local LLM)

The job-match percentage is assigned by the AI, but in a way that stays explainable and stable. The LLM acts as a judge on **each JD requirement individually**, using only retrieved evidence; the percentage is computed from its verdicts with a fixed formula. This is more reliable than asking a 7B model for one holistic number, which tends to vary between runs and cannot be audited.

- 1 For each JD requirement, retrieve the top resume evidence spans (Phase 4/5) and related-skill chunks from RAG (e.g. "PostgreSQL experience is relevant to a MySQL requirement").
- 2 The LLM returns, per requirement: {verdict: met | partial | transferable | missing, confidence: low|med|high, evidence_ids, rationale}. The validator rejects any verdict that cites a non-existent evidence ID or claims "met" with no evidence.
- 3 The LLM also rates two holistic items on anchored 1-5 rubrics: seniority/scope fit and responsibilities alignment, each with cited evidence.
- 4 Run the judge 3 times (fixed seeds, temperature 0 for the first, small temperature for the others) and take the majority verdict per requirement; variance becomes the confidence band.
- 5 **AI Match %** = weighted aggregation: requirement verdict values (met 1.0, partial 0.6, transferable 0.4, missing 0) weighted by required (×3) vs. preferred (×1), combined with the two holistic ratings, using the same dimension weights as the hybrid table in Phase 4 so the two scores are comparable.
- 6 **Cross-check:** compare with the Hybrid Match score. If they differ by more than 15 points, show a "low agreement" badge and list the requirements where the verdicts disagree, so the user sees why.

Displayed as: **AI Match 72% (±5)** — "8 of 11 required skills met, 2 partial, 1 missing. Hybrid score: 68%." Each requirement row expands to show the verdict, rationale and the exact resume lines used. The UI states that this is the app's own AI-assisted estimate of how well the resume evidences the JD, not a prediction of hiring outcome.

12B. JD-based feedback on how to improve the resume

Tasks

- 1 Classify gaps per requirement from 12A: missing, partial, transferable-but-unstated, claimed-but-not-evidenced, and satisfied-but-buried (evidence exists but only deep in the resume or in the wrong section).
- 2 **Estimate uplift** for each actionable gap: re-run the deterministic aggregation with that requirement's verdict hypothetically improved to see how many points it is worth. Only gaps the candidate can truthfully address (surface existing evidence, reword, reorder, add a user-confirmed fact) are ranked as quick wins; true skill gaps route to Phase 13.
- 3 Generate section-level feedback (summary, skills, experience, projects, education) with the LLM from the gap list, evidence and RAG skill relations, e.g. "Your Airflow work at X satisfies the 'data pipeline orchestration' requirement but appears only in bullet 4 of your second role; move it up and name the tool."
- 4 Link each feedback item to one-click rewrite generation (Phase 8) and, for real gaps, to project recommendations (Phase 13).
- 5 Combine with the Readability Reviewer: when a JD is given, the 6-second snapshot is also checked against the JD's top 3 requirements.
- 6 Build POST /api/recommendations/jd; extend GET /api/analysis/{id}/match with ai_match_percentage, confidence_band, per_requirement_verdicts, hybrid_score and agreement.

Exit criteria: identical inputs yield an AI Match % within ± 3 points across repeated runs; every requirement verdict carries evidence IDs or an explicit "no evidence found"; the agreement between AI Match % and human fit ratings is measured in Phase 15 and compared with the hybrid score; every feedback item links to a JD requirement and resume evidence (or its absence).

Phase 13 — Skill-Gap Analysis & Project Recommendation Engine

Tasks

- 1 Build the normalized skill graph/taxonomy connecting resume skills, JD requirements, and RAG-sourced skill relationships.
- 2 Classify each JD requirement as: missing from resume, preferred-but-missing, related/partially transferable, claimed-but-not-evidenced, or satisfied — each linked to the exact JD requirement and resume evidence (or its absence). **Reuse the Phase 12 verdicts rather than recomputing.**
- 3 Build the project-recommendation engine: for each significant gap cluster, generate a recommendation grounded in the RAG knowledge base with fields — title, why recommended (specific JD gaps addressed), skills demonstrated, suggested stack, deliverables, resume evidence the candidate could legitimately write after completion, difficulty (with defined criteria), and scope.
- 4 Ensure recommendation copy never implies that completing a project guarantees qualification for the role.
- 5 Build POST /api/projects/recommend. **In no-JD mode, gaps come from the inferred role's norms (Phase 11).**

Exit criteria: every recommendation cites RAG source identifiers; missing/partial/claimed-but-unevidenced skills are visibly distinguished in the output.

Phase 14 — LaTeX/Overleaf Source Analysis

Tasks

- 1 Accept .tex files and .zip LaTeX projects; parse source commands, sections, tables, columns, and hyperlinks directly from source (more reliable than parsing compiled PDF output alone).
- 2 Where a safe, sandboxed LaTeX environment is available, compile the source to extract layout-accurate text and detect formatting-risk elements (multi-column layouts, tabular environments, tikz/graphics-heavy content).
- 3 Feed both source-level and compiled-output signals into the existing ATS structural checks (Phase 2), the Readability layout metrics (Phase 10) and the resume schema (Phase 1), rather than building a parallel pipeline.
- 4 Enforce strict sandboxing for compilation: no network access, resource/time limits, no execution of arbitrary shell-escape LaTeX packages.
- 5 **Detect LaTeX-template placeholders (e.g. untouched template macros and sample text) with the Phase 9B detector.**

Exit criteria: .tex / .zip uploads produce the same canonical resume schema output as PDF/DOCX uploads; compilation sandbox blocks shell-escape and network access in a security test.

Phase 15 — Research Experiments, Ablations & Human Evaluation

[EXTENDED]

Tasks

- 1 Run the ablation study comparing five configurations: (A) TF-IDF baseline, (B) embedding-only, (C) LLM-only, (D) RAG+LLM, (E) full hybrid system. Report which components move which metrics — do not assume the full system wins on every axis.
- 2 **Readability ablation:** RRI with deterministic metrics only vs. + RAG role norms vs. + LLM skim test; report each variant's correlation with recruiter ratings.
- 3 **AI Match ablation:** single holistic LLM score vs. per-requirement LLM verdicts vs. hybrid score; report run-to-run variance and correlation with human fit labels.
- 4 Run the full evaluation plan (Section 9) across extraction, matching, RAG, rewriting, readability, placeholder detection, general/JD recommendations and the AI-likeness module.
- 5 Build the human-evaluation rubric: reviewers score resume feedback on factual correctness, usefulness, specificity, actionability, JD relevance, and whether rewrites preserve the candidate's original facts. Use multiple reviewers and report inter-rater agreement where feasible.
- 6 Write up the research contribution: a hybrid, privacy-preserving architecture separating deterministic ATS/structural analysis from semantic matching and LLM reasoning, grounded through a role/skill/project RAG layer, with evidence-validated rewriting, **a role-aware human-readability framework (RRI), and evidence-bound LLM-as-judge match scoring** — evaluated against conventional and LLM-only baselines.

Exit criteria: ablation results and baseline comparisons are documented with limitations stated explicitly; human-evaluation inter-rater agreement is reported, not assumed.

Phase 16 — UI Polish, Privacy Controls, Export & Documentation

[EXTENDED]

Tasks

- 1 Finish the remaining frontend pages: landing/upload, parsing preview, analysis dashboard, ATS analysis, **readability review**, job-match analysis (**hybrid + AI Match %**), **general recommendations**, **JD-based feedback**, skill-gap view, AI-likeness & placeholder view, rewrite editor, project recommendations, evidence/source panel, settings/model/runtime page, export report.
- 2 Add privacy controls: explicit data-retention settings, a clear "processed entirely on-device" statement, and a delete-my-data action.
- 3 Build POST /api/export for the final report/export (PDF or shareable document of scores, evidence, warnings, accepted rewrites, and recommendations).
- 4 Write user-facing documentation and developer docs (docs/, README.md, models/README.md) covering setup on the target laptop, model swapping, and the scoring methodology of **all three rubrics (ATS, RRI, AI Match %)**.
- 5 Final end-to-end hardening pass: error states, empty states, large-file handling, **LLM-busy/queued states**, **VRAM-exhaustion fallback**, and a full run-through of the 16-step workflow (Section 7) on representative resumes, with and without JDs.

Exit criteria: all frontend pages function end-to-end; exported report accurately reflects on-screen results; documentation lets a new developer set up and run the system on the target laptop without out-of-band help.

Future Scope Phases [FUTURE SCOPE]

These phases are fully specified so they can be scheduled after the core product is stable. Both depend on Phases 3–7 and 12. Both introduce new risks (network access, third-party content, code execution), so each has its own safety gate.

Phase F1 — Live Matching Job Descriptions from Trusted Sources

[FUTURE SCOPE]

Privacy model

This is the first feature that needs the network, so it is **off by default and opt-in per search**. The resume never leaves the device. Only a derived, editable query leaves: normalized job titles, top skills, location/remote preference and seniority — shown to the user before it is sent. Fetched postings are parsed and scored locally.

Trusted sources

"Trusted" means a source that publishes postings through an official API or feed and permits this use. Plan to integrate, in priority order: (1) employer applicant-tracking-system public job boards that expose documented read APIs (e.g. Greenhouse Job Board API, Lever Postings API, Ashby public job board API), queried for an allowlist of companies; (2) aggregator APIs with developer terms that allow this use (e.g. Adzuna; USAJOBS for US federal roles; Remotive/Arbeithub for remote roles); (3) region-specific boards only where an official partner API is available. Large boards that prohibit scraping in their terms (e.g. LinkedIn, Indeed, Naukri) are **not** scraped; they can be added only through an official partnership. Each source's terms, rate limits and attribution requirements are recorded in a source registry and reviewed before enabling.

Tasks

- 1 Build a livejobs/ connector interface (search(query) → [RawPosting]) with one adapter per source, rate limiting, caching, and a source registry (terms URL, allowed use, attribution, trust tier).
- 2 Normalize postings through the Phase 3 JD parser; store as LiveJobPosting with source, URL, posted_at, fetched_at, expiry, and content hash.

- 3 Quality & trust filters: dedupe across sources (content hash + fuzzy title/company match), drop postings older than a configurable freshness window, verify the apply link resolves to the employer or source domain, and flag scam heuristics (payment requests, personal-email contacts, unrealistic pay, missing company).
- 4 Retrieval: embed postings into a separate local vector index; retrieve candidates by resume-profile embedding + filters (location, remote, seniority); rerank with the Hybrid Match score (fast, deterministic).
- 5 Run the Phase 12 AI Match judge only on the top N (e.g. 10) reranked postings to stay within the laptop's LLM budget; queue the work and stream results in as they finish.
- 6 Build POST /api/jobs/live/search, GET /api/jobs/live/{search_id}; UI: query preview/consent dialog, results list with source badge, posted date, Hybrid and AI Match %, top matched/missing requirements, and "Analyze against this JD" which feeds the posting into the normal JD flow.
- 7 Scheduled refresh (optional, user-enabled) with notification of new high-match postings; all cached postings expire and are purged automatically.

LLM / RAG role: Local vector index of fetched postings for retrieval; LLM only for AI Match % on the top-N shortlist. Fetched posting text is treated as untrusted data (prompt-injection suite extended to postings).

Exit criteria: no request leaving the device contains resume text or personal identifiers (verified by a network-capture test); every displayed posting has a source, posted date and working link; precision@10 of relevance judged by humans on a sample set is reported; expired postings are never shown.

Phase F2 — JD-Based Practice Assessment from Company Question History [FUTURE SCOPE]

Sourcing "previous history of questions" responsibly

Verbatim interview questions collected from sites such as Glassdoor or LeetCode discussion threads are generally covered by those sites' terms and by copyright, and some are shared in breach of candidates' confidentiality agreements. The plan therefore stores **question history as structured topic records**, not copied question text, and generates original practice questions from those records.

- **Permitted sources:** company-published preparation material and sample questions; openly licensed datasets; user-contributed reports submitted through ResumeIQ with consent and moderation; and in-house authored questions.
- **Stored per record:** company, role family, round type (online assessment, technical, system design, behavioral, HR), topic/skill, question pattern (e.g. "sliding-window string problem", "design a rate limiter"), difficulty, reported date, source, verification level (official / multiple reports / single report).
- **Fallback:** when a company has little or no history, generate from the JD's requirements and the role's norms, and label the test "JD-based" rather than "company-based".

Tasks

- 1 Build the knowledge_base/question_bank/ collection and ingestion pipeline (normalize, dedupe by pattern embedding, attach provenance and verification level); build a contribution + moderation flow for user reports.
- 2 Test blueprint: from the JD (Phase 3) and the company's history records (RAG, filtered by company + role family + recency), compute a deterministic blueprint — number of questions per round type, topic and difficulty — weighted by how often each topic appears in the history and how central it is to the JD.

- 3 Question generation: the LLM writes original questions per blueprint slot, conditioned on the retrieved pattern records, with answer keys, rubrics and explanations. MCQs are validated by a second, independent LLM pass that must reach the same answer; coding questions are generated with reference solutions and test cases that must pass in the sandbox before the question is accepted. Near-duplicate checks prevent reproducing any stored source text.
- 4 Test engine: timed sections, MCQ, short answer, coding (in-browser editor), and behavioral (STAR-format) questions; autosave; results stored as AssessmentAttempt.
- 5 Grading: MCQ and coding are graded deterministically (answer key; test cases in a sandboxed runner such as a self-hosted Judge0 or a locked-down container with no network, CPU/memory/time limits). Short-answer and behavioral answers are graded by the LLM against the rubric, with per-criterion feedback, clearly marked as AI feedback.
- 6 Results feed back into the product: weak topics become skill gaps in Phase 13 and learning resources are recommended via RAG.
- 7 Build POST /api/assessments/generate, GET /api/assessments/{id}, POST /api/assessments/{id}/submit, GET /api/assessments/{id}/result; UI shows provenance ("topics reported for this company in 2025–26, 4 reports") next to each section.

LLM / RAG role: RAG over the question-history corpus and role norms drives the blueprint and generation; LLM generates questions and grades open-ended answers; deterministic graders for MCQ and code.

Exit criteria: no generated question is a near-duplicate of stored source text; $\geq 95\%$ of MCQs pass independent answer verification in a sampled human review; every coding question's reference solution passes its tests in the sandbox; the code runner blocks network and filesystem escape in a security test; UI states that tests are practice material and not the questions the candidate will receive.

8. API & Data Model Reference

8.1 Backend API

Endpoint	Purpose	Phase
POST /api/resume/upload	Upload and parse resume	1
POST /api/jd/analyze	Parse job description	3
POST /api/analysis/run	Run complete analysis (with or without JD)	2
GET /api/analysis/{id}	Retrieve analysis	2
GET /api/analysis/{id}/ats	ATS results	2
GET /api/analysis/{id}/match	Hybrid match + AI Match %, per-requirement verdicts, agreement	4, 12
POST /api/readability/run	Run the Readability Reviewer (ATS 2.0)	10
GET /api/analysis/{id}/readability	RRI, dimensions, 6-second snapshot, bullet grades	10
GET /api/analysis/{id}/quality	Placeholder / lorem ipsum, AI-likeness and weak-claim findings	9
POST /api/recommendations/general	Recommendations without a JD	11
POST /api/recommendations/jd	JD-based improvement feedback	12
POST /api/rewrite	Generate a grounded rewrite	8
POST /api/projects/recommend	Generate gap-based projects	13

Endpoint	Purpose	Phase
GET /api/knowledge/search	Debug/inspect RAG retrieval	6
GET /api/jobs/{job_id}/events	SSE progress stream for queued LLM work	0
GET/PUT /api/settings/runtime	Model/runtime settings	7
POST /api/export	Generate final report/export	16
POST /api/jobs/live/search · GET /api/jobs/live/{id}	Live matching JDs (opt-in)	F1
POST /api/assessments/generate · GET /api/assessments/{id} · POST .../submit · GET .../result	Practice assessments	F2

8.2 Core Data Model

Entity	Key fields
Resume	id, filename, parsed_text, structured_json, layout_metadata, created_at
JobDescription	id, title, raw_text, structured_json, source (upload paste live)
Analysis	id, resume_id, jd_id (nullable), target_role, ats_score, readability_score, match_score, ai_match_percentage, versions (rubric, prompt, model, corpus)
Skill	canonical_name, aliases, category
Evidence	analysis_id, source_type, span, claim, confidence
Finding	analysis_id, module, category, severity, span, message, remediation
ReadabilityReport	analysis_id, rri_score, dimension_scores, snapshot_inferred, snapshot_full, bullet_grades, confidence, rri_version
MatchJudgment	analysis_id, requirement_id, verdict, confidence, evidence_ids, rationale, run_votes
Recommendation	type (general jd project), text, source_ids, evidence_ids, priority, status
KnowledgeChunk	source_id, collection, content, embedding, metadata, version
Rewrite	original, suggestion, evidence_ids, validator_result, accepted
LiveJobPosting (F1)	id, source, url, company, title, posted_at, fetched_at, expires_at, content_hash, structured_json, trust_flags
QuestionRecord (F2)	company, role_family, round_type, topic, pattern, difficulty, reported_at, source, verification_level
Assessment / AssessmentAttempt (F2)	blueprint, questions, answer_keys · answers, scores, per_criterion_feedback, started_at, submitted_at

Every endpoint that returns a score, warning, or recommendation should also return the Evidence / source_ids that support it — this is the API-level enforcement of the evidence-grounding requirement running through every phase.

9. Cross-Cutting Evaluation Plan

Treat ResumeIQ as an NLP/AI system to evaluate, not only a website to test. Run the ablation study and the human-evaluation rubric described in Phase 15 before declaring any component "done." Measure, don't assume, that the full system outperforms simpler baselines on every metric.

Component	Metrics
Resume section/field extraction	Precision, recall, F1
Skill extraction	Precision, recall, F1
JD requirement classification	Precision, recall, F1
Semantic matching (hybrid)	MRR, Recall@K, NDCG@K, ranking correlation where labels exist
AI Match % [new]	Run-to-run variance; per-requirement verdict accuracy vs. human labels; correlation with human fit ratings; agreement with hybrid score
ATS structural checks	Rule accuracy against manually annotated resumes
Readability Reviewer (RRI) [new]	Spearman correlation with recruiter ratings; inter-rater agreement of the panel; bullet-grade accuracy vs. human grades; reproducibility
Placeholder / lorem ipsum [new]	Recall on placeholder fixtures; false-positive rate on clean resumes
General & JD-based recommendations [new]	Human rating for correctness, usefulness, actionability; citation correctness; % items with evidence
LLM feedback	Human rating for factuality, usefulness, specificity, groundedness
Rewriting	Human preference, factual preservation, JD relevance
RAG (incl. role norms)	Retrieval Recall@K, groundedness/citation correctness
AI-likeness module	Precision/recall/F1 on a held-out labeled set; calibration where possible
Live JDs (F1)	Human-judged precision@10, freshness, dead-link rate, scam-filter precision
Assessments (F2)	Answer-key correctness, difficulty calibration vs. observed scores, near-duplicate rate, grading agreement with humans
System performance	Latency, VRAM usage, RAM usage, throughput on the i7 / RTX 4060 8 GB / 24 GB target

10. Key Risks & Mitigations

Risk	Mitigation
7B-8B LLM is too slow or runs out of VRAM on the RTX 4060 8 GB	Benchmark multiple models/quantizations early (Phase 7); keep the LLM interface swappable; serialize calls; move embedder/reranker to CPU, then fall back to a 3B model or shorter context
LLM invents facts, metrics, or skills not in the resume	Hard rewrite-validator gate (Phase 8) that rejects unsupported claims before they reach the user; automated fabrication tests in CI
AI Match % is unstable or unexplainable	Per-requirement verdicts instead of one holistic number; fixed seeds, majority vote, confidence band; deterministic aggregation; hybrid cross-check with disagreement badge
Readability rubric is subjective or doesn't reflect real reviewers	Anchored rubric with examples; recruiter validation panel with inter-rater agreement; weights tuned on data and versioned; disclosure as the app's own rubric
Placeholder detector flags legitimate text (e.g. real bracketed terms)	Deterministic high-precision rules first; LLM confirmation only for ambiguous spans; measured false-positive rate on clean resumes
AI-likeness module is over-claimed as an authorship detector	Enforce confidence-band language and precision/recall reporting from Phase 9 onward; UI copy review before each release

Risk	Mitigation
JD, resume, posting, or question-bank content attempts prompt injection	Treat all uploaded/retrieved content as untrusted data (Section 2.2); injection test suite in Phase 7, extended to F1/F2 content
LaTeX compilation is a code-execution surface	Strict sandboxing (no network, resource/time limits, no shell-escape) as a hard requirement in Phase 14
Scores are mistaken by users for industry standards or hiring predictions	Persistent UI disclosure that ATS, RRI and AI Match % are the application's own transparent, versioned rubrics
Evaluation (Phase 15) is expensive/time-consuming and gets skipped	Scope a minimal-but-real evaluation set from the start; treat evaluation as an acceptance-criteria gate, not a stretch goal
F1: job sources change terms/APIs, or postings are stale or fraudulent	Source registry with terms review; adapters isolated per source; freshness window; link verification; scam heuristics
F1: privacy leak when searching online	Opt-in, query preview, only derived non-identifying terms sent; network-capture test in CI
F2: copyright/confidentiality of interview questions	Store topic/pattern records not verbatim text; generate original questions; near-duplicate check; provenance shown; moderation of contributions
F2: code-runner is a code-execution surface	Sandboxed runner with no network, strict CPU/memory/time limits, and a security test before launch

11. Consolidated Acceptance Criteria

- A PDF with standard sections parses with no critical extraction errors.
- ATS score is reproducible for the same input, and component scores sum to the total.
- Readability Index is reproducible for the same input and configuration, every finding cites a resume span (and a role-norm source where role-specific), and its correlation with recruiter ratings is measured and reported.
- Placeholder and lorem-ipsuim text is detected and reported as critical in every fixture that contains it.
- A resume without a JD produces a prioritized, evidence-linked general recommendation report.
- Every major job-match conclusion has evidence from the resume/JD.
- AI Match % is stable within ± 3 points across repeated runs, shows per-requirement verdicts with evidence, and is displayed alongside the hybrid score.
- Missing skills are distinguished from related/partial matches.
- Rewrites cannot introduce unsupported facts without an explicit user-provided fact.
- RAG recommendations include source identifiers.
- The local LLM completes the core feedback workflow on the i7 / RTX 4060 8 GB / 24 GB laptop with peak VRAM below 7 GB.
- The user can inspect why each warning or recommendation was generated.
- AI-likeness output is probabilistic/qualified and never claims authorship certainty.
- Evaluation reports compare multiple baselines and state limitations.
- *Future scope*: live-JD search sends no personal data off-device; practice tests contain no verbatim source questions and are labelled as practice.

12. Milestones [NEW]

The phases group into five milestones. Each milestone ends with a working end-to-end path the user can try.

Milestone	Phases	What the user can do at the end
M1 — Deterministic core	0, 1, 2, 3, 4	Upload a resume and JD; see parser preview, ATS score and a hybrid match score with evidence
M2 — Local AI foundation	5, 6, 7	Same flow, now with real embeddings, a searchable knowledge base and a benchmarked local LLM
M3 — Grounded AI features	8, 9, 10	Rewrites, AI-likeness and placeholder findings, and the Readability Reviewer (ATS 2.0)
M4 — Recommendations & AI Match	11, 12, 13, 14	Full no-JD and JD-based recommendation reports, AI Match %, project suggestions, LaTeX support
M5 — Evaluation & release	15, 16	Evaluated, documented, privacy-controlled release with export
Future — F1, F2	F1, F2	Opt-in live job matches and JD/company-based practice tests

13. Immediate Next Steps

- 1 Scaffold the repository per Section 6 and set up CI (lint, unit tests, a smoke test hitting `/api/resume/upload`).
- 2 Collect/anonymize the fixture set of resumes (clean, column-heavy, table-heavy, image-heavy, corrupt, placeholder/lorem-ipsum, generic) and JDs (terse, verbose, nested).
- 3 On the target laptop, install Ollama or llama.cpp and run a quick smoke test of Qwen2.5-7B-Instruct Q4_K_M at 8k context to confirm VRAM headroom before Phase 5/7 benchmarking.
- 4 Kick off Phase 1 (PDF/DOCX ingestion, including layout metadata) — it is the dependency for every later phase.
- 5 Stand up the non-negotiable-constraints test suite from Section 2.2 (fabrication checks, prompt-injection checks, AI-likeness confidence-language checks, score-disclosure checks) now, so every later phase is built against it rather than retrofitted.
- 6 Start recruiting 3+ recruiters/hiring managers for the Readability Reviewer validation panel; their ratings are needed by Phase 10.
- 7 Begin drafting the role-norms collection for the 3–5 role families you most want to support first.